

Seminar Presentation Script: Observer Pattern

Group 52

Nguyen Dinh Minh Huy (Student ID: 25125083)

July 11, 2026

Presenters

- **Speaker A:** Member A
- **Speaker B:** Member B

Presentation Flow & Dialogue

Slide 1: Title Slide (The Observer Design Pattern)

Speaker A: Hello everyone, and welcome to Group 52's presentation. Today, my colleague and I will be discussing the Observer Design Pattern—a fundamental behavioral pattern that plays a critical role in decoupling state changes and notifications in modern software. Let's start with a real-world scenario.

Slide 2: Real-world Problem (Bank Account Notifications)

Speaker B: Imagine you are building a modern financial core banking system. We have a `BankAccount` class where customers perform transactions, like depositing money. When a deposit occurs, several peripheral systems need to react immediately. For instance, the User Interface needs to update the balance display. The Email system needs to send a confirmation message. A security log needs to record the transaction. An AI Fraud Detector needs to scan for suspicious activity. The mobile app needs to send a push notification, and the analytics system needs to register a deposit event. How can we notify all these systems dynamically at runtime without tangling our core business logic?

Slide 3: Naive Solution (Without Pattern)

Speaker A: In a naive solution, we might create concrete classes for each system: `UI`, `EmailSystem`, `Logger`, `FraudDetector`, `MobileApp`, and `Analytics`. Our `NaiveBankAccount` class then maintains references to all these concrete classes. As shown in this code snippet, inside the `deposit()` method, the bank account explicitly calls methods on all six objects: updating the UI, sending email alerts, logging details, running fraud checks, pushing notifications, and recording metrics. This works, but it is a maintenance nightmare.

Slide 4: Drawbacks of the Naive Solution

Speaker B: Indeed, this naive approach has three severe flaws:

First, **Tight Coupling**. The `BankAccount` is tightly coupled to six concrete classes. It has to know their names, structures, and specific update methods.

Second, it violates the **Open-Closed Principle (OCP)**. If compliance requirements demand a new Credit Score or Marketing promotion check system, we must modify the core `BankAccount` class.

Lastly, it is **rigid and hard to scale**. We cannot easily unsubscribe or disable notifications dynamically at runtime.

Slide 5: Introduction to the Observer Pattern

Speaker A: To solve this coupling issue, we use the **Observer Pattern**. The pattern defines a one-to-many dependency between objects. When the Subject—our `BankAccount`—changes its state via a deposit, all its dependents—our systems/Observers—are notified and updated automatically. The key is that the `BankAccount` only interacts with an abstract interface, not the concrete classes. This cleanly separates the core transaction logic from the notification channels.

Slide 6: General Class Diagram

Speaker B: Here is the general UML diagram of the pattern. We have:

1. The **Subject** interface, which defines methods to attach, detach, and notify observers.
2. The **Observer** interface, which defines a simple `update()` or `onTransaction()` method.
3. The **ConcreteSubject**, which maintains the list of Observers and broadcasts notifications.
4. **ConcreteObservers**, which implement the `update()` method to perform their custom action.

Slide 7: Specific Class Diagram (Bank Account Example)

Speaker A: Applying this to our problem, we introduce the abstract `AccountObserver` interface, which declares a single `onTransaction(type, amount, balance)` method. Our `BankAccount` class implements the Subject role. It maintains a list of `AccountObserver` pointers. The bank account doesn't know or care if it's notifying the UI, the Logger, the Fraud AI, or an Analytics tracker. It simply loops through the pointers and invokes `onTransaction()`.

Slide 8: Code Implementation & Demo

Speaker B: Let's look at the refactored code. The `BankAccount` class is now extremely clean. To add a notification channel, we call `attach()`; to remove it, we call `detach()`. Adding a new compliance observer requires absolutely zero changes to the bank account code itself! In our console demo, you can see that when a deposit is made, all six observers are successfully notified. When we dynamically detach the Email and Mobile App observers, the subsequent deposit only triggers logs, UI, fraud detection, and analytics updates.

Slide 9: Pros & Cons

Speaker A: Like any design choice, the Observer pattern has trade-offs. The main **Pros** are that it satisfies the Open-Closed Principle, separates concerns cleanly, and allows dynamic runtime changes. The **Cons** include the **Lapsed Listener Problem**—if observers are destroyed without detaching, the subject will hold dangling references, leading to memory leaks or crashes. Additionally, developers must be careful about ordering concerns and avoiding infinite loops if observers trigger state updates back on the subject.

Slide 10: Other Real-world Applications

Speaker B: In modern software development, you see this pattern everywhere. In web dev, browser DOM events like `addEventListener` are classic Observers. Similarly, state management libraries like Redux or Vuex notify UI components to re-render when the state store updates. In mobile dev, reactive frameworks like RxSwift, Android LiveData, and SwiftUI's `@Published` variables use Observers to keep UI views synchronized with underlying data models.

Slide 11: Interactive Quiz

Speaker A: Now, to wrap things up and keep you all awake, we have a short quiz! Correct answers will get small sweet treats, so raise your hand fast!

Speaker B: [Reads Q1] “What type of design pattern is the Observer Pattern?” ... [Wait for hands] ... Correct! It is Behavioral. Here is your candy!

Speaker A: [Reads Q2] “Which SOLID principle is satisfied when adding new subscribers without changing the channel code?” ... Yes! Open-Closed Principle. Excellent.

Speaker B: [Reads Q3] “What problem occurs when a listener is deleted without unsubscribing?” ... Correct, the Lapsed Listener problem leading to memory leaks.

Slide 12: Conclusion & References

Speaker A: In conclusion, the Observer pattern is a powerful tool to build scalable, loosely-coupled event-driven systems. Thank you all for listening. We are now open to any questions or reviews from the audience.